CÓDIGOCLARO · RUTA KOTLIN

Tema 09 · Herencia e interfaces

Cuaderno de estudio para DAM/DAW · 10 lecciones

Qué vas a aprender

open, override, abstract, interface y sealed class.

Este cuaderno trabaja herencia e interfaces en pasos cortos. La meta no es repetir palabras de memoria: es reconocer el problema, elegir una herramienta adecuada, explicar por qué funciona y comprobarla con datos distintos. Reserva una sesión para leer el modelo, otra para resolver el ejercicio y una tercera para intentar el laboratorio sin mirar la solución.

Resultados de aprendizaje

- Explicar con tus palabras qué problema resuelve herencia e interfaces y en qué parte de una aplicación DAM/DAW aparece.
- Leer el ejemplo, predecir el resultado y justificar cada decisión relevante.
- Resolver una práctica, comprobar casos normales y límite, y depurar un fallo con un método reproducible.

Cómo estudiar este tema

Ten a mano un editor compatible con la tecnología y copia los ejemplos en un archivo nuevo. Antes de ejecutar, escribe tu predicción. Si no coincide con la salida, no borres el intento: anota qué supuesto era incorrecto y prueba una variación mínima. Al terminar, explica el concepto a otra persona usando un ejemplo propio.

Punto de control inicial

Antes de avanzar, escribe dos ideas que ya conoces sobre este tema y una pregunta que todavía no sabes responder. Vuelve a ellas al final de la lección 10. Si cambias de respuesta, explica qué ejemplo o prueba hizo que cambiaras de opinión.

Lección 1 · El modelo mental

Idea central

Las clases Kotlin son final por defecto. open permite heredar y override marca la implementación que reemplaza un miembro abierto. Una interface declara capacidades que varias clases pueden implementar, mientras una clase abstracta puede compartir estado y comportamiento. Una jerarquía sealed limita los subtipos directos permitidos y permite que when compruebe casos conocidos. La composición suele ser preferible cuando los objetos solo colaboran, sin ser variantes del mismo tipo.

Para razonar sobre herencia e interfaces, separa cuatro preguntas: qué datos existen al empezar, qué regla se aplica, qué cambia durante el proceso y qué resultado debe observar quien usa el programa. Esta separación ayuda a no confundir la sintaxis con la finalidad. Una línea puede compilar y aun así resolver mal el problema si el dato representa otra cosa o falta un caso.

En Kotlin, ten presente este marco

El itinerario usa Kotlin/JVM: el compilador infiere muchos tipos, pero el programa sigue teniendo comprobación estática. val impide reasignar el nombre, aunque el objeto referenciado pueda ser mutable; var permite reasignar. Los tipos nullable se escriben con ? y obligan a tratar la ausencia. La biblioteca distingue interfaces de solo lectura de interfaces mutables.

Comprueba tu comprensión

- ¿Qué parte de la explicación define la regla principal de herencia e interfaces?
- ¿Qué dato puede cambiar y qué dato debería permanecer estable?
- ¿Qué observación concreta demostraría que tu programa cumple el objetivo?

Lección 2 · Vocabulario y reglas

Reglas de Kotlin que afectan a este tema

Las propiedades pueden generar getters y setters según su declaración. data class aporta operaciones basadas en sus parámetros principales, como copy y destructuring. La herencia requiere declarar open; las interfaces describen contratos. sealed class limita las subclases conocidas y permite comprobaciones exhaustivas con when.

Ideas específicas de herencia e interfaces

Un objeto reúne datos y operaciones que deben permanecer coherentes. Encapsular no significa ocultarlo todo: significa proteger las invariantes y ofrecer operaciones claras. Composición permite construir un comportamiento usando colaboradores; herencia expresa una relación de sustitución y no solo la reutilización de código. Prueba la creación del objeto y sus estados válidos e inválidos.

Glosario de trabajo

Término	Significado en este recorrido
Clase	Definición usada para crear objetos con un contrato común.
Instancia	Objeto concreto creado según una clase.
Invariante	Regla que el objeto debe mantener en sus estados válidos.
Composición	Construcción de un objeto a partir de colaboradores.

Relaciona cada término con una línea del ejemplo o una decisión del ejercicio. Comprueba las diferencias de versión en la fuente oficial enlazada antes de aplicar una regla al proyecto.

Lección 3 · Lectura guiada del ejemplo

Primero identifica la entrada, luego sigue el valor por el programa. No supongas que una asignación transforma el dato: a veces solo enlaza un nombre con el mismo objeto. Observa dónde empieza cada bloque, qué condición activa una rama y qué instrucción produce la salida. Si el ejemplo tiene varias líneas, ejecútalo por partes y añade temporalmente observaciones seguras para ver los valores intermedios.

Código · kotlin

```kotlin
interface Etiquetable { fun etiqueta(): String }
open class Documento(val nombre: String) : Etiquetable {
    override fun etiqueta() = nombre
}
class Informe(nombre: String, val paginas: Int) : Documento(nombre)

fun main() = println(Informe("resumen", 3).etiqueta())
```

Salida esperada

```
resumen
```

Registro de la traza

Momento	Qué anotar
Antes de ejecutar	Valores iniciales, sus tipos y qué operación esperas que ocurra.
Después de cada paso	Nombre afectado, valor nuevo y regla que produjo el cambio.
Al encontrar una decisión	Condición evaluada y camino que se ejecutó.
Al terminar	Salida real, diferencia respecto a tu predicción y causa.
Con una entrada distinta	Qué resultado cambia y qué parte del contrato permanece igual.

Traza en papel

- Antes de la primera instrucción: apunta los valores iniciales y sus tipos.
- Después de cada operación: anota solo los nombres que hayan cambiado y la razón.
- Al final: compara tu predicción con «resumen» y explica la diferencia si la hay.

Cambia un dato de entrada y vuelve a hacer la traza. Una buena explicación debe anticipar el nuevo resultado, no limitarse a repetir la salida original.

Lección 4 · Decidir cuándo usarlo

Un objeto reúne datos y operaciones que deben permanecer coherentes. Encapsular no significa ocultarlo todo: significa proteger las invariantes y ofrecer operaciones claras. Composición permite construir un comportamiento usando colaboradores; herencia expresa una relación de sustitución y no solo la reutilización de código. Prueba la creación del objeto y sus estados válidos e inválidos.

Caso de uso

Un cargador puede devolver datos o un error, y la interfaz de usuario debe describir ambas variantes de forma exhaustiva.

Compara antes de elegir

- ¿La solución más directa deja claro el propósito o requiere conocer demasiados detalles internos?
- ¿Qué coste aparece si crece el volumen de datos o se añade un nuevo caso?
- ¿Qué comprobación automatizada detectaría una modificación accidental?

Enfoques relacionados

- Modelar las variantes con sealed class.
- Usar sealed interface cuando las variantes no necesiten compartir estado.

No existe una alternativa mejor en todos los contextos. Elige una y escribe su ventaja y su límite para el tamaño, la versión y los requisitos del ejercicio. Si dos enfoques cumplen exactamente el mismo contrato y ambos son legibles, pueden considerarse válidos.

Lección 5 · Errores y diagnóstico

Fallos que conviene reconocer

- Intentar heredar de una clase final sin declararla open. Para localizarlo, reduce el caso a la menor entrada que lo reproduce, observa el valor implicado y verifica la regla antes de cambiar el código.
- Olvidar override al implementar un miembro abierto. Para localizarlo, reduce el caso a la menor entrada que lo reproduce, observa el valor implicado y verifica la regla antes de cambiar el código.
- Crear jerarquías profundas cuando una interfaz o composición bastaría. Para localizarlo, reduce el caso a la menor entrada que lo reproduce, observa el valor implicado y verifica la regla antes de cambiar el código.

Rutina de depuración

1. Describe el resultado esperado y el resultado real sin mezclar ambos. 2. Reproduce el fallo con una entrada fija. 3. Reduce el programa hasta dejar la primera operación que se comporta distinto. 4. Formula una hipótesis comprobable y cambia una sola cosa. 5. Añade una prueba que impida que el mismo fallo vuelva. Este orden también funciona cuando el error aparece en una API, una interfaz o un archivo.

Preguntas de revisión

- ¿El error está en los datos, en la regla o en cómo se presenta el resultado?
- ¿Qué caso límite no estaba cubierto por el ejemplo inicial?
- ¿La solución corrige la causa o solo oculta el síntoma?

Lección 6 · Práctica guiada

Situación de DAM/DAW

Un cargador puede devolver datos o un error, y la interfaz de usuario debe describir ambas variantes de forma exhaustiva.

Enunciado

Representa un resultado sellado con Éxito(valor) y Fallo(mensaje), y crea una función que describa ambos casos.

Convierte el enunciado en requisitos verificables

- Escribe qué entradas acepta la solución y qué debe devolver o mostrar.
- Enumera las reglas de negocio y separa las que son obligatorias de las decisiones de presentación.
- Antes de programar, completa la tabla de pruebas y predice cada resultado.

Casos que debe superar

Entrada	Resultado esperado
Éxito(3)	"valor 3"
Fallo("sin datos")	"error: sin datos"
Éxito(0)	"valor 0"

Qué demuestra cada caso

Clasifica cada fila de la tabla como caso habitual, límite o variación importante. Si el enunciado no define qué hacer con una entrada fuera de contrato, no inventes un resultado silencioso: anota la ambigüedad y propone una regla antes de programar. En la práctica profesional, acordar esa regla evita que dos implementaciones parezcan correctas y se comporten de forma incompatible.

Plan de solución

- Divide la tarea en pasos pequeños que puedan comprobarse por separado.
- Implementa primero el caso habitual y después añade los límites y errores previstos.
- Ejecuta cada caso tras un cambio; no esperes al final para descubrir qué parte falló.

Lección 7 · Solución explicada

Una solución no se justifica solo porque la salida coincida en un ejemplo. Debe respetar el contrato para todas las entradas admitidas. Lee el código siguiente y vincula cada operación con una frase del enunciado. Si no puedes señalar esa relación, revisa el diseño antes de continuar.

Código · kotlin

```kotlin
sealed interface Resultado
data class Exito(val valor: Int) : Resultado
data class Fallo(val mensaje: String) : Resultado

fun describir(resultado: Resultado): String = when (resultado) {
    is Exito -> "valor " + resultado.valor
    is Fallo -> "error: " + resultado.mensaje
}
```

Criterio de aceptación: Se acepta sealed class en lugar de sealed interface si when cubre todos los subtipos definidos.

Por qué se sigue este razonamiento

Resultado es sealed y tiene dos variantes con datos: Exito guarda un entero y Fallo un mensaje. El when comprueba el subtipo y construye el texto correspondiente; por eso el éxito con 3, el fallo con mensaje y el éxito con 0 se describen sin una rama else. Al ser sellado, el compilador puede comprobar que se cubren las variantes conocidas.

Comprobación de corrección

Comprueba cada caso de la tabla y explica qué parte de la solución lo cubre. Después revisa el caso mínimo, el máximo razonable y una entrada inválida si el contrato la permite. Comprueba también los tipos de entrada y salida: una coincidencia textual puede ocultar un valor mal tipado o una conversión con pérdida.

Paso	Justificación que debes poder dar
Contrato	Qué valores acepta el ejercicio y qué resultado promete.
Regla	Qué operación transforma la entrada en la salida.
Pruebas	Qué caso comprueba lo normal y cuál descubre un límite.

Alternativas válidas

- Modelar las variantes con sealed class.
- Usar sealed interface cuando las variantes no necesiten compartir estado.

Al comparar soluciones, prioriza contrato correcto, legibilidad y pruebas. Una alternativa que cambia la representación interna puede ser válida; una que cambia silenciosamente el significado de la salida no lo es.

Lección 8 · Laboratorio de aplicación

Retoma el ejercicio de herencia e interfaces y amplíalo como una pequeña funcionalidad de una aplicación de gestión académica. Conserva el requisito original y añade una regla nueva que tenga sentido para los datos. Antes de modificar el código, escribe la regla en lenguaje natural, un ejemplo que la cumpla y otro que la incumpla.

Variación para transferir lo aprendido

Añade una tercera variante a Resultado y observa que el when debe incorporar una rama para seguir siendo exhaustivo.

Desafíos progresivos

- Nivel 1: añade un caso límite que no aparecía en la tabla y registra el resultado esperado.
- Nivel 2: cambia la forma de recibir o recorrer los datos sin cambiar el contrato externo.

- Nivel 3: separa la regla principal de su entrada/salida y prueba ambas partes de forma independiente.

Criterios de entrega

- La solución conserva el resultado esperado del ejercicio original.
- El nuevo comportamiento queda cubierto por una prueba que falla antes del cambio.
- Los nombres permiten seguir el flujo sin comentarios que repitan la sintaxis.
- La entrada inválida o el error externo tiene una respuesta definida.

Prueba de regresión

Guarda los casos originales antes de añadir la regla nueva. Ejecuta la batería completa después de cada cambio, no solo el nuevo caso. Si una prueba deja de pasar, decide si encontraste un defecto o si cambió el contrato y hay que actualizar la expectativa con una explicación. Termina con una nota breve: qué cambió, qué comprobaste y qué límite sigue abierto.

Anota qué parte se puede reutilizar en otro módulo y qué parte depende del contexto de la aplicación. Esa distinción ayuda a decidir si merece la pena extraer una función o componente.

Lección 9 · Síntesis y fuentes

Resumen del tema

Has estudiado herencia e interfaces desde el modelo conceptual hasta una práctica comprobable. La secuencia de trabajo es: identificar entradas y contrato, elegir la operación adecuada, seguir el flujo, comprobar casos límite y conservar una prueba para la siguiente modificación. Aplica las reglas de Kotlin sin confundir una característica del lenguaje con una API concreta del entorno.

Lista de control antes de avanzar

- Puedo explicar el concepto con un ejemplo propio y predecir su salida.
- Puedo identificar al menos un caso límite y un error habitual.
- Puedo comparar dos enfoques y justificar mi elección.

- Puedo verificar una solución sin depender de mirar el resultado una sola vez.

Fuentes oficiales para ampliar

JetBrains / Kotlin — Inheritance · JetBrains / Kotlin — Sealed classes and interfaces

Consulta la sección de la fuente que corresponda a la versión de tu proyecto. Este cuaderno resume conceptos para estudiar; cuando haya una diferencia de detalle, prevalece la documentación oficial del lenguaje o biblioteca.

Lección 10 · Repaso activo

Cierra el material y responde sin copiar frases. Si dudas, vuelve a la página indicada, comprueba la fuente primaria y reescribe la respuesta con tus palabras. La recuperación activa ayuda a detectar qué parte aún depende de memorizar un ejemplo concreto.

Preguntas

- ¿Qué problema aborda herencia e interfaces y qué dato necesita?
- ¿Qué diferencia hay entre una solución que funciona en el ejemplo y una que cumple el contrato?
- ¿Qué error frecuente te parece más probable y cómo lo reproducirías?
- ¿Qué alternativa elegirías en un proyecto pequeño y por qué?

Respuestas de comprobación

- La idea clave se resume en esta explicación: Las clases Kotlin son final por defecto. open permite heredar y override marca la implementación que reemplaza un miembro abierto. Una interface declara capacidades que varias clases pueden implementar, mientras una clase abstracta puede compartir estado y comportamiento. Una jerarquía sealed limita los subtipos directos permitidos y permite que when compruebe casos conocidos. La composición suele ser preferible cuando los objetos solo colaboran, sin ser variantes del mismo tipo.
- Una fuente inicial de fallo descrita en el tema es: Intentar heredar de una clase final sin declararla open.
- Una alternativa es válida si mantiene las entradas, salidas y reglas acordadas y supera las mismas pruebas.